

KNN (**K** Nearest Neighbors)

دکتر مبصر

هاتف جانعلی پور

درس هوش مصنوعی

تاریخ (31/2/1405)

مقدمه ای بر ماشین لرنینگ

در علم یادگیری ماشین (Machine Learning)، به موضوع طراحی ماشین‌هایی پرداخته می‌شود که با استفاده از مثال‌های داده شده به آن‌ها و تجربیات خودشان، بیاموزند. در واقع، در این علم تلاش می‌شود تا با بهره‌گیری از الگوریتم‌ها، یک ماشین به شکلی طراحی شود که بدون آنکه صراحتاً برنامه‌ریزی و تک‌تک اقدامات به آن دیکته شود بتواند بیاموزد و عمل کند. در یادگیری ماشین، به جای برنامه‌نویسی همه چیز، داده‌ها به یک الگوریتم عمومی داده می‌شوند و این الگوریتم است که براساس داده‌هایی که به آن داده شده منطق خود را می‌سازد. یادگیری ماشین روش‌های گوناگونی دارد که از آن جمله می‌توان به یادگیری نظارت شده، نظارت نشده و یادگیری تقویتی اشاره کرد. الگوریتم‌های مورد استفاده در یادگیری ماشین جزو این سه دسته هستند.

یادگیری نظارت‌شده (Supervised Learning)

اغلب روش‌های یادگیری ماشین از یادگیری نظارت‌شده استفاده می‌کنند. در یادگیری ماشین نظارت‌شده، سیستم تلاش می‌کند تا از نمونه‌های پیشینی بیاموزد که در اختیار آن قرار گرفته. به عبارت دیگر، در این نوع یادگیری، سیستم تلاش می‌کند تا الگوها را بر اساس مثال‌های داده‌شده به آن فرا بگیرد. داده‌های آموزشی شامل ورودی (ویژگی‌ها) و خروجی صحیح (برچسب یا مقدار هدف) هستند. مدل یاد می‌گیرد که نگاشت ورودی به خروجی را پیدا کند.

زیرشاخه‌ها:

- طبقه‌بندی (Classification): خروجی گسسته (مثل تشخیص اسپم/غیر اسپم، نوع گل).
- رگرسیون (Regression): خروجی پیوسته (مثل پیش‌بینی قیمت خانه، دما).

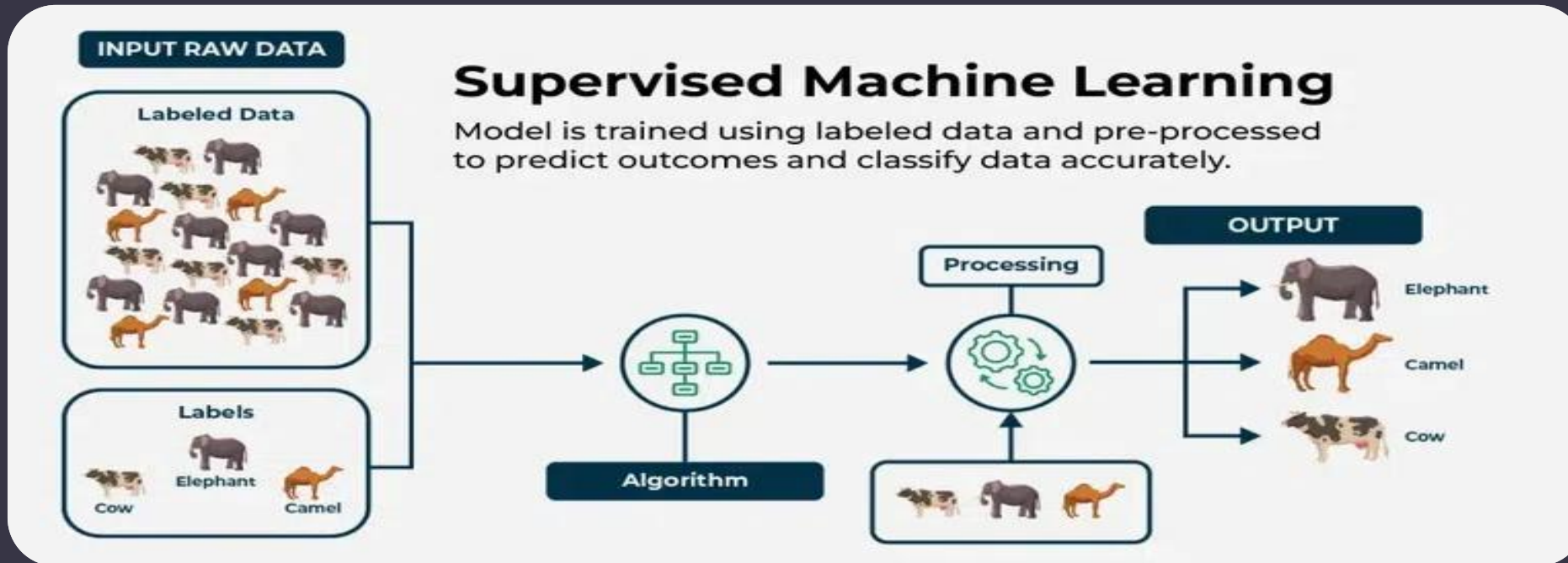
الگوریتم‌های معروف:

KNN، رگرسیون خطی، درخت تصمیم، جنگل تصادفی، SVM، شبکه عصبی.

مثال عملی:

پیش‌بینی اینکه فردا باران می‌بارد یا نه (طبقه‌بندی) – یا مقدار دقیق بارش (رگرسیون).

مواردی مانند وضعیت هوا شامل ابری بودن یا نبودن، آفتابی بودن یا نبودن، وجود یا عدم وجود مه، بارش یا عدم بارش باران و تاریخ بازی از جمله ویژگی‌هایی هستند که وضعیت یک مسابقه فوتبال را توصیف می‌کنند. حال اگر در این مجموعه داده به عنوان مثال، ستونی وجود داشته باشد که مشخص کند برای هر نمونه داده در شرایط جوی موجود برای آن نمونه خاص بازی فوتبال انجام شده یا نشده (برچسب‌ها) اصطلاحاً می‌گوییم مجموعه داده برچسب‌دار است. اگر آموزش الگوریتم از چنین مجموعه داده‌ای استفاده شود و به آن آموخته شود که بر اساس نمونه داده‌هایی که وضعیت آن‌ها مشخص است (بازی فوتبال انجام شده یا نشده)، درباره نمونه داده‌هایی که وضعیت آن‌ها نامشخص است تصمیم‌گیری کند، اصطلاحاً گفته می‌شود یادگیری ماشین نظارت شده است.



یادگیری بدون نظارت (Unsupervised Learning)

در یادگیری نظارت نشده، الگوریتم باید خود به تنهایی به دنبال ساختارهای جالب موجود در داده‌ها باشد. به بیان ریاضی، یادگیری نظارت نشده مربوط به زمانی است که در مجموعه داده فقط متغیرهای ورودی (X) وجود داشته باشند و هیچ متغیر داده خروجی موجود نباشد. به این نوع یادگیری، نظارت نشده گفته می‌شود زیرا برخلاف یادگیری نظارت شده، هیچ پاسخ صحیح داده شده‌ای وجود ندارد و ماشین خود باید به دنبال پاسخ باشد. به بیان دیگر، هنگامی که الگوریتم برای کار کردن از مجموعه داده‌ای بهره گیرد که فاقد داده‌های برچسب‌دار (متغیرهای خروجی) است، از مکانیزم دیگری برای یادگیری و تصمیم‌گیری استفاده می‌کند. به چنین نوع یادگیری، نظارت نشده گفته می‌شود. یادگیری نظارت نشده قابل تقسیم به مسائل خوشه‌بندی و انجمنی است.

خوشه‌بندی (Clustering) : گروه‌بندی داده‌های مشابه (مثل دسته‌بندی مشتریان بر اساس رفتار).
کاهش ابعاد (Dimensionality Reduction) : فشردن داده با حفظ اطلاعات مهم (مثل PCA، t-SNE)
یافتن قاعده همبستگی (Association Rule) : کشف روابط بین ویژگی‌ها (مثل «کسانی که نان می‌خرند، کره هم می‌خرند»).

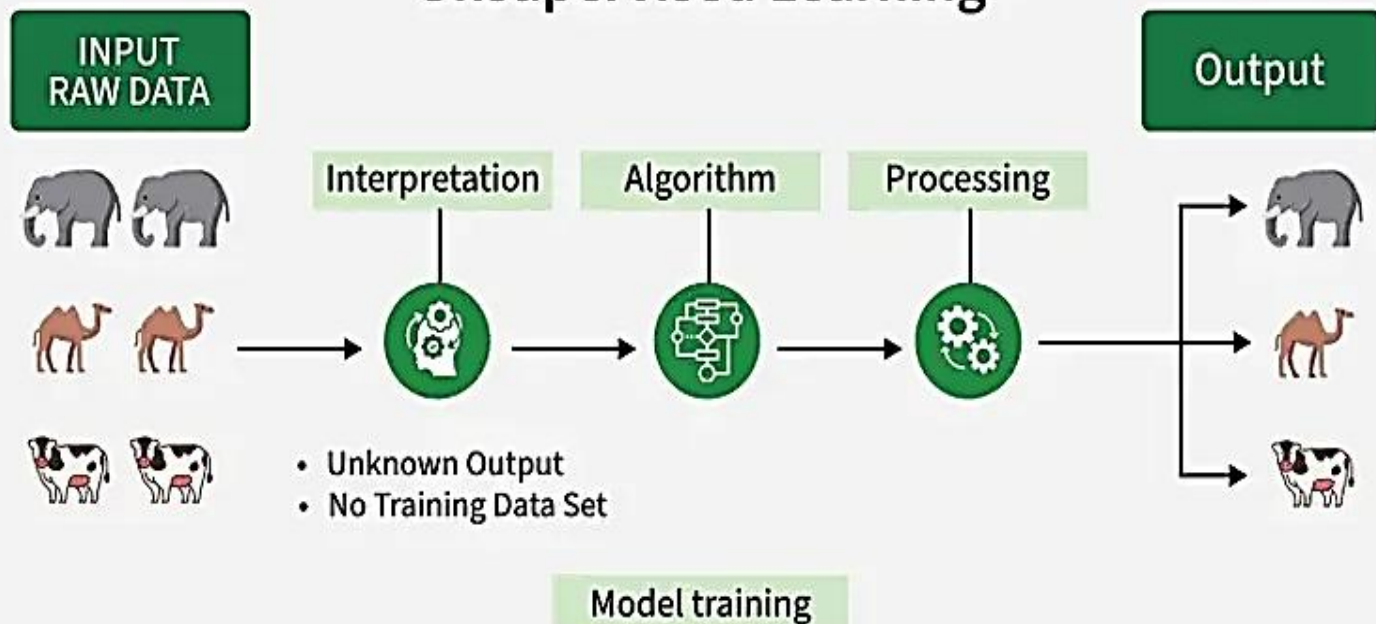
الگوریتم‌های معروف:

Autoencoder، Apriori، PCA، DBSCAN، K-Means

مثال عملی:

دسته‌بندی اخبار بدون برچسب به گروه‌های موضوعی (ورزش، سیاست، فناوری).

Unsupervised Learning



- خوشه‌بندی (Clustering): گروه‌بندی داده‌های مشابه (مثل دسته‌بندی مشتریان بر اساس رفتار).

- کاهش ابعاد (Dimensionality Reduction): فشرده‌سازی داده با حفظ اطلاعات مهم (مثل PCA، t-SNE).

- یافتن قاعده همبستگی (Association Rule): کشف روابط بین ویژگی‌ها (مثل «کسانی که نان می‌خرند، کره هم می‌خرند»).

الگوریتم‌های معروف:
Autoencoder، Apriori، PCA، DBSCAN، K-Means

مثال عملی:
دسته‌بندی اخبار بدون برچسب به گروه‌های موضوعی (ورزش، سیاست، فناوری).

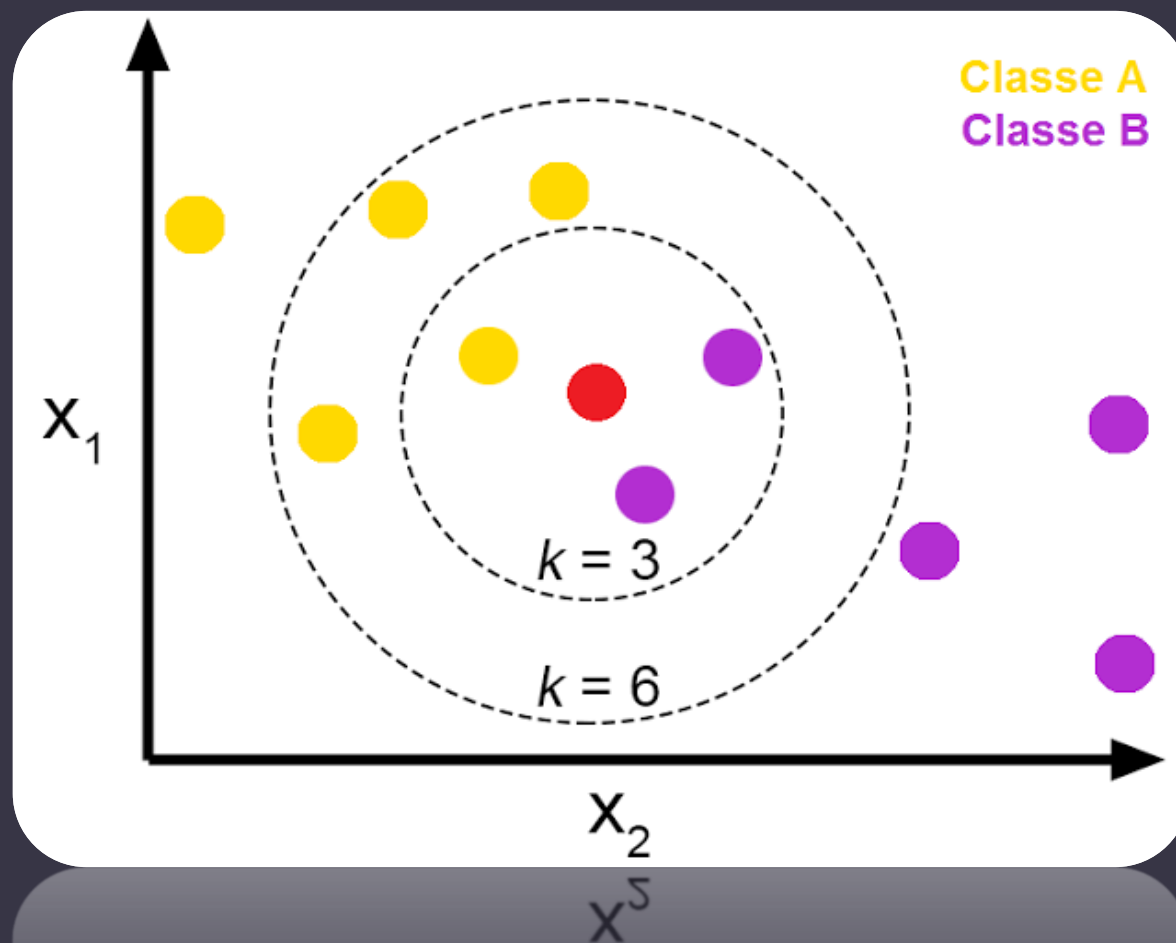
KNN چیست ؟

KNN یک الگوریتم یادگیری نظارت شده (Supervised Learning) است که هم برای طبقه‌بندی (Classification) و هم برای رگرسیون (Regression) استفاده می‌شود. اساس کار آن «همسایگی» است: داده جدید بر اساس نزدیک‌ترین نمونه‌های موجود در مجموعه آموزشی، پیش‌بینی می‌شود.

نحوه عملکرد (گام به گام)

1. عدد K (تعداد همسایگان) را انتخاب کنید.
 2. فاصله داده جدید را از تمام داده‌های آموزشی محاسبه کنید (معمولاً فاصله اقلیدسی).
 3. K داده‌ای که کمترین فاصله را دارند به عنوان همسایگان انتخاب کنید.
- برای طبقه‌بندی: رأی‌گیری اکثریت – کلاسی که بیشترین تکرار را در بین همسایگان دارد، به داده جدید اختصاص می‌یابد.
- برای رگرسیون: میانگین مقادیر هدف همسایگان را به عنوان خروجی در نظر بگیرید.

انتخاب مقدار K



- K کوچک (مثلاً 1): بیش‌برازش (Overfitting) حساس به نویز.
- K بزرگ: کم‌برازش (Underfitting) مرزهای تصمیم خیلی ساده می‌شوند.
- روش معمول: آزمایش K های مختلف و استفاده از اعتبارسنجی متقابل (Cross-Validation)
- نکته: معمولاً K را فرد انتخاب می‌کنند تا در رأی‌گیری مساوی نشود.

Distance functions

Euclidean

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

Manhattan

$$\sum_{i=1}^k |x_i - y_i|$$

Minkowski

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$

معیارهای فاصله

- اقلیدسی (Euclidean) متداول ترین گزینه.

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- منهتن (Manhattan) جمع قدر مطلق اختلافات.

- مینکوفسکی (Minkowski) حالت کلی که با پارامتر p کنترل می شود ($p=2$ اقلیدسی، $p=1$ منهتن).

توضیح خطوط کلیدی کد وارد کردن کتابخانه‌های مورد نیاز

python Copy Download

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
from collections import Counter
```

```
from collections import Counter
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.datasets import load_iris
```

- **numpy** : برای عملیات عددی (گرچه در این کد زیاد مستقیم استفاده نشده، اما پشت‌صحنه توسط sklearn نیاز است).
- **matplotlib.pyplot** : برای رسم نمودار و نمایش همسایگی‌ها.
- **load_iris** : تابعی برای بارگذاری دیتاست معروف Iris (از بنق).
- **NearestNeighbors** : کلاسی از scikit-learn که برای یافتن k همسایه نزدیک بهینه و سریع استفاده می‌شود (با پشتیبانی از درخت‌های kd-tree یا ball-tree)
- **StandardScaler** : برای نرمال‌سازی داده‌ها (میانگین صفر، واریانس یک) - در KNN بسیار حیاتی است چون فاصله تحت تأثیر مقیاس ویژگی‌هاست.
- **Counter** : از collections برای شمارش آسان کلاس‌ها در رأی‌گیری نهایی.

بارگذاری داده و انتخاب ویژگی‌ها

python

Copy Download

```
iris = load_iris()
X = iris.data[:, :2] # فقط دو ویژگی اول: طول و عرض کاسبرگ
y = iris.target
feature_names = iris.feature_names[:2]
target_names = iris.target_names
```

```
target_names = iris.target_names
```

- `iris.data` یک آرایه 150×4 است (۴ ویژگی). ما فقط دو ستون اول (۲:) را می‌گیریم تا بتوانیم در فضای دوبعدی رسم کنیم.
- `iris.target` برچسب کلاس (۰، ۱، ۲) و `target_names` نام کلاس‌ها (مثلاً 'setosa') است.
- `feature_names` برای برچسب محورهای نمودار استفاده می‌شود.

Standardization (نرمال سازی داده ها)

- `fit_transform` ابتدا میانگین و انحراف معیار هر ویژگی را روی داده `X` محاسبه می کند (`fit`) و سپس تبدیل را اعمال می کند (`transform`).

نتیجه: هر ویژگی دارای میانگین ۰ و انحراف معیار ۱ می شود. این کار باعث می شود فاصله اقلیدسی معنی دار باشد.

python

 Copy  Download

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

```
X_scaled = scaler.fit_transform(X)  
scaler = StandardScaler()
```

تعریف نقطه جدید و مقیاس سازی آن

نقطه جدید را با همان ویژگی‌ها (طول کاسبرگ ۵,۲، عرض کاسبرگ ۳,۶) تعریف می‌کنیم. شکل (۱,۲) برای یک نمونه با دو ویژگی.

حتماً باید همین scaler که روی داده آموزشی fit شده را برای تبدیل نقطه جدید استفاده کنیم تا در فضای یکسان قرار گیرد.

python

Copy Download

```
new_point = np.array([[5.2, 3.6]])  
new_point_scaled = scaler.transform(new_point)
```

```
new_point_scaled = scaler.transform(new_point)  
new_point = np.array([[5.2, 3.6]])
```

یافتن ۵ همسایه نزدیک

NearestNeighbors یک مدل بدون آموزش (چون lazy learning است) ایجاد می کند که فقط داده را نگه می دارد.

n_neighbors=5 تعداد همسایگان مورد نظر.

metric='euclidean' فاصله اقلیدسی.

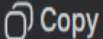
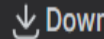
kneighbors برای نقطه پرس و جو (query)

point فاصله تا نزدیک ترین ها و شاخص های آنها را

برمی گرداند.

خروجی: distances آرایه ای از فواصل (مرتب صعودی)

و indices آرایه ای از اندیس های متناظر در (X_scaled)

python  Copy  Download

```
k = 5
nbrs = NearestNeighbors(n_neighbors=k, metric='euclidean').fit(X_scaled)
distances, indices = nbrs.kneighbors(new_point_scaled)
```

```
distances, indices = nbrs.kneighbors(new_point_scaled)
nbrs = NearestNeighbors(n_neighbors=k, metric='euclidean').fit(X_scaled)
```

k = 5

نمایش نتایج در کنسول

python

Copy Download

```
print("نقطه جدید:", new_point[0])
print("همسایه نزدیک (شاخصها) ۵:", indices[0])
print("فاصله‌ها:", distances[0])
print("کلاس همسایگان", y[indices[0]])
```

```
print("شاخص نزدیک:", indices[0])
print("فاصله‌ها:", distances[0])
print("کلاس همسایگان:", y[indices[0]])
```

indices[0] چون neighbors برای یک نقطه خروجی با ابعاد (k, 1) می‌دهد، اندیس سطر اول را می‌گیریم.

با استفاده از y[indices[0]] برچسب کلاس واقعی همسایگان را می‌بینیم.

توصیه قلبی

دنیای ماشین لرنینگ امروز در همه جای زندگی ما نفوذ کرده است: از سیستم توصیه گر فیلم در نتفلیکس گرفته تا تشخیص تقلب بانکی، خودروهای خودران و حتی تشخیص بیماری از روی تصاویر پزشکی. این حوزه نه فقط یک تکنولوژی، بلکه یک روش تفکر است: روشی که در آن به جای برنامه نویسی صریح قوانین، به ماشین اجازه می دهیم از داده ها «یاد بگیرد».

اما این دنیا بی چالش نیست. با وجود الگوریتم های قدرتمند (مانند KNN، جنگل تصادفی، شبکه های عصبی عمیق)، هیچ «نقره ای» ای وجود ندارد که همه مسائل را حل کند. موفقیت در ماشین لرنینگ نیازمند درک عمیق از ریاضیات آمار، مهندسی ویژگی ها، پیش پردازش داده، انتخاب مدل مناسب، ارزیابی صحیح و مهم تر از همه تشخیص مسئله کسب و کاری است که می خواهید حل کنید.

دنیای ماشین لرنینگ بسیار شیرین و عمیق است، اما سرشار از شایعات و مسیرهای کاذب (مثل «در یک هفته متخصص شو»). هیچ جایگزینی برای تمرین عملی وجود ندارد.

از همین امروز یک نوت بوک باز کنید، دیتاست Iris را بارگذاری کنید (همان که در پروژه KNN دیدیم)، و شروع کنید به تغییر Kها، رسم نمودارهای مختلف و دیدن تأثیر پارامترها. با هر پروژه کوچک، اعتماد به نفس شما بیشتر می شود.

یادگیری ماشین لرنینگ یک سفر است، نه مقصد. از هر اشتباهی در ارزیابی مدل یا انتخاب ویژگی، یک درس می گیرید. اگر عاشق حل مسئله و کار با داده هستید، این راه برای شما ساخته شده است.

موفق و پیروز باشید.

sources

[NumPy](#)

[matplotlib](#)

[Sklearn\(datasets\)](#)

[Sklearn neighbors\(KNN\)](#)

[Collections](#)

[Python](#)

[Faradars](#)

[DeepSeek](#)

[W2school](#)

GitHub : Hatef-skywalker007
